

Team Juggernauts

Custom Navigation Stack Design & Zone Nav Decoupling

A* Planner · PD / Pure-Pursuit Controller · Waypoint Sequencer

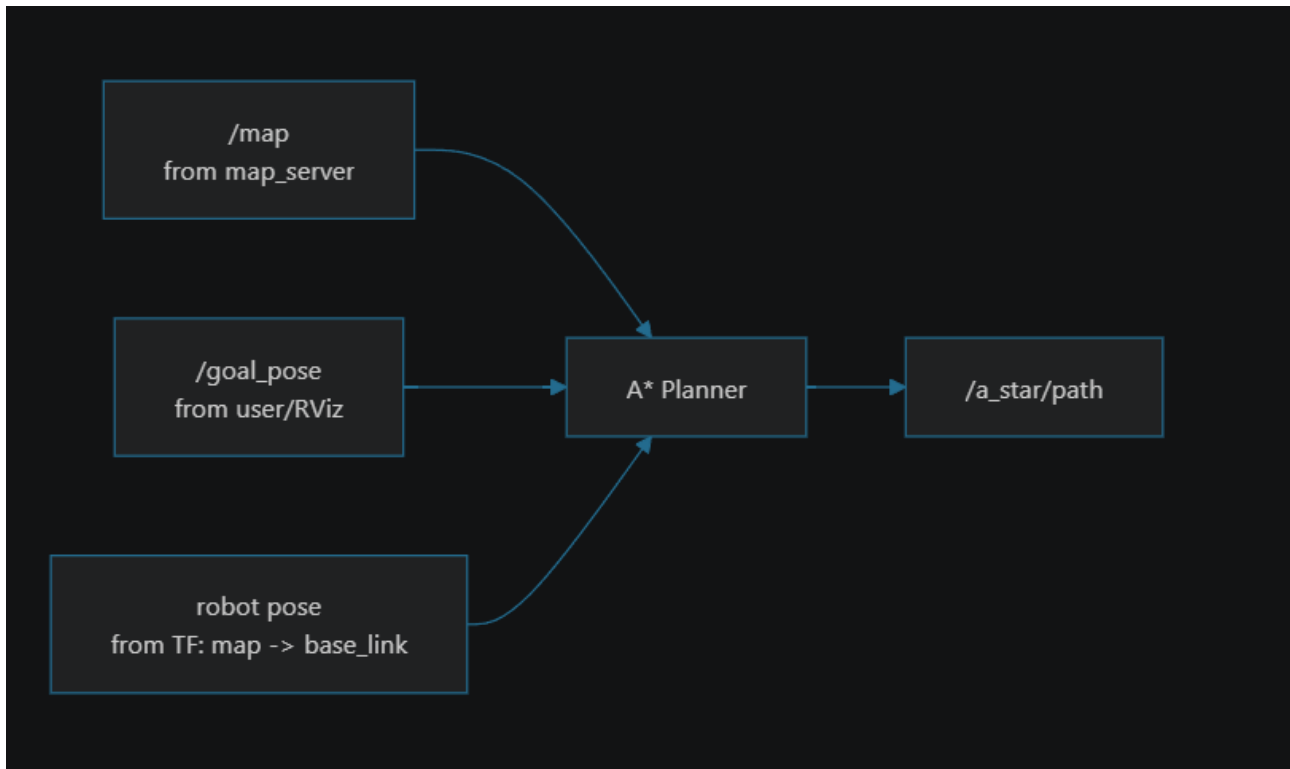
This guide covers: the custom A*+PD/pure-pursuit navigation architecture · verified findings on why `diy_zone_nav` is already fully decoupled from it · two real bugs found and fixed along the way · the new `diy_waypoint_sequencer` package for automated goal sequencing, including its real functional test results.

Companion reference: [docs/reuse_plan_step1.md](#) Step 13 (the full investigation and verification detail behind this document).

1 · Architecture Overview

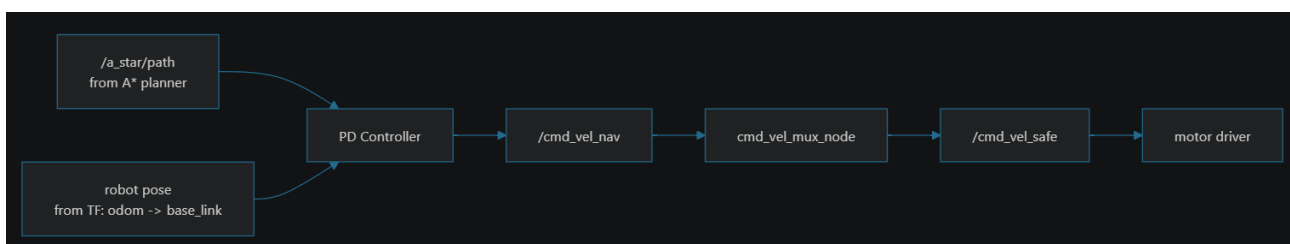
This robot uses **only nav2_map_server + nav2_lifecycle_manager** from the Nav2 ecosystem — map serving only, not the full Nav2 navigation system (no planner_server, controller_server, or costmap layers). Path planning and control are fully custom.

Planning Pipeline



Source: teammate-provided architecture diagram.

Control Pipeline



Source: teammate-provided architecture diagram.

VERIFIED — /cmd_vel_nav is the SAME topic name cmd_vel_mux_node already reads for AUTONOMOUS mode — no mux changes were needed to plug this controller in.

2 · The Zone Nav Question — What Was Actually Verified

The ask: design so `diy_zone_nav` can be removed entirely with no impact, since the plan is for the controller to eventually consume "info from the zone navigator." Investigated the real code before designing anything — did not assume.

Check	Method	Result
Does <code>diy_planning/diy_motion_planner</code> reference <code>/nav_mode</code> , <code>/speed_limit</code> , or <code>zone_nav</code> anywhere?	<code>grep -rn</code> across both packages	Zero hits — no existing coupling at all
Does the A* planner crash/hang with no <code>/goal_pose</code> ?	Read <code>goal_callback</code> 's guard clauses directly	No — early-return, idles safely
Does <code>cmd_vel_mux</code> 's AUTONOMOUS mode depend on <code>zone_nav</code> ?	Read <code>cmd_vel_mux_node.py</code> 's mode logic	No — reads <code>/cmd_vel_nav</code> unconditionally; BLIND_DRIVE only via <code>zone_nav</code> 's own explicit service call
Does <code>zone_nav</code> crash if Nav2's costmap/controller services don't exist?	Read the service-call sites directly	No — already checks <code>service_is_ready()</code> first, skips gracefully

VERIFIED — `diy_zone_nav` was ALREADY fully orphaned and safely removable in this architecture before any code was changed. Its `/speed_limit` output and costmap `SetParameters` calls have no consumer and no valid service target respectively, in a stack that only runs `nav2_map_server`. Nothing needed to be built to satisfy "can take zone navigator out" for the pipeline as it already existed.

3 · Two Real Bugs Found and Fixed While Verifying This

Bug 1 — cmd_vel_topic Launch Default Mismatch

pd_navigation.launch.py and pure_pursuit_navigation.launch.py declared cmd_vel_topic defaulting to /cmd_vel (the simulation value), while their sibling arguments (base_frame, use_sim_time) already defaulted to hardware values (base_link, false).

WARNING — A bare hardware launch with no override would silently publish velocity commands into /cmd_vel, which cmd_vel_mux never subscribes to — the robot simply would not move, with no error anywhere.

VERIFIED — FIXED: both launch files now default cmd_vel_topic to /cmd_vel_nav, consistent with their other hardware defaults.

Bug 2 — No ROS Signal for "Goal Reached"

Neither pd_motion_planner_node.py nor pure_pursuit_motion_planner_node.py published anything when the robot reached its goal — only an internal log line. This blocks any future automated sequencing (nothing to react to).

VERIFIED — FIXED: added a /pd/goal_reached (std_msgs/Bool) publisher to both controller nodes, published at the exact point each already detects goal completion — purely additive, no existing behavior changed.

4 · New Package: diy_waypoint_sequencer

Built to satisfy the actual open design question: something needs to auto-publish `/goal_pose` in sequence for a competition run where no human is clicking RViz goals.

Key Design Decision

NOTE — Built as a brand-new, standalone, minimal package — NOT added inside `diy_zone_nav`, `diy_planning`, or `diy_motion_planner`. Putting it inside `diy_zone_nav` would have re-coupled "can I remove `zone_nav`" with "do I still get automated sequencing" — exactly the ambiguity this whole task was about avoiding.

How It Works

- Loads a simple waypoints YAML (label, x, y, yaw) — deliberately NOT reusing `zone_waypoints.yaml`'s much richer zone/radius/mode schema, which encodes Nav2-costmap concepts this simpler stack doesn't use.
- Waits for `/green_light` (same competition start-trigger topic convention already used by `zone_nav`) — or starts immediately after a configurable delay for bench testing.
- Publishes the current waypoint on `/goal_pose`.
- Advances to the next waypoint when `/pd/goal_reached` fires True.
- Optionally loops back to the first waypoint after the last (loop: true).

VERIFIED — If this node is not run: `/goal_pose` is simply never auto-published — a human can still publish it manually (RViz "2D Goal Pose", or `ros2 topic pub`) with zero code changes anywhere else in the stack.

Verification — Real Functional Test Results

A real functional test (not just written and assumed correct) — `tf_transformations` is not installed in this sandbox and there is no sudo access to add it, so its one function was stubbed with equivalent pure math for the test only.

#	Behavior Tested	Result
1	No goal published before <code>/green_light</code>	PASS
2	First waypoint published correctly on green light	PASS
3	<code>/pd/goal_reached</code> advances to the next waypoint	PASS
4	Sequence stops cleanly after the last waypoint (no loop)	PASS
5	Duplicate <code>/green_light</code> after start is a no-op	PASS
6	<code>wait_for_green_light:=false</code> auto-starts with no green light at all	PASS
7	<code>loop:=true</code> wraps back to waypoint 0 after the last	PASS

VERIFIED — Also confirmed the package builds clean in a full 15-package workspace rebuild, and its launch file arguments resolve correctly.

5 · How to Launch / Test This Stack

Hardware, PD controller, with automated sequencing

```
ros2 launch diy_motion_planner pd_navigation.launch.py \
  map_yaml:=/absolute/path/to/map.yaml
```

```
ros2 launch diy_waypoint_sequencer waypoint_sequencer.launch.py \
  waypoints_file:=/absolute/path/to/waypoints.yaml
```

Manual goal testing (no sequencer needed)

```
ros2 topic pub --once /goal_pose geometry_msgs/msg/PoseStamped \
  "{header: {frame_id: map}, pose: {position: \
  {x: 1.0, y: 1.0}, orientation: {w: 1.0}}}"
```

NOTE — `diy_zone_nav` is NOT required for any of the above to work.

Switching maps during testing

The course map (2D occupancy grid, `.pgm/.yaml` — feeds `nav2_map_server` → `/map` → the A* planner) is expected to change often as testing progresses. `map_yaml` in both `pd_navigation.launch.py` and `pure_pursuit_navigation.launch.py` now defaults to, in priority order: (1) the `DIY_MAP_YAML` env var if set, else (2) `$DIY_ROS_WS/src/DIY-Challenge-Repo/maps/course_traced_smooth.yaml` (the current final course map — hand-traced borders smoothed via `preprocess_map.py`; see `docs/reuse_plan_step1.md` Steps 19-24 for the full history).

Switching maps — no launch-file edits needed

```
# Once per test session, before launching anything:
export DIY_MAP_YAML=/absolute/path/to/new_map.yaml
```

```
# Or override for a single run without exporting anything:
ros2 launch diy_motion_planner pd_navigation.launch.py \
  map_yaml:=/absolute/path/to/new_map.yaml
```

NOTE — `map_yaml` (2D occupancy grid, `nav2_map_server/A*`) and `map_pcd_path` (3D point cloud, `map_localizer's map`→`odom` TF — see `jetson_bringup_guide.md` §2/§3) are two separate, unrelated maps. A `.pgm/.yaml` pair cannot be used as a `map_pcd_path` value or vice versa.

Generating real waypoints for controller testing

`diy_waypoint_sequencer/config/waypoints.yaml` ships with placeholder `x/y/yaw` values only meant to document the schema — not real, map-valid coordinates. `scripts/pick_waypoints.py` is a click-to-pick tool (same pattern as `scripts/register_zones_to_map.py's` `--pick` mode) that generates a real `waypoints.yaml` by clicking points directly on the actual map you are testing against.

Pick waypoints on the real test map

```
python3 scripts/pick_waypoints.py \
  --map maps/course_traced_smooth.yaml \
  --output maps/test_waypoints.yaml \
  --overlay maps/test_waypoints_preview.png
```

Click waypoints in the order the robot should visit them; each click is checked against the map's own occupancy data immediately (a click on an occupied/unknown pixel prints a warning right away — press 'u' to undo and click again). Heading (yaw) is computed automatically as the bearing toward the next waypoint — override with `--uniform-yaw <radians>` for a fixed heading everywhere. `--overlay` saves a PNG with numbered markers + heading arrows to sanity-check the path before running it on the robot.

NOTE — The tool also prints a ready-to-copy `initial_x/initial_y/initial_yaw` override matching `wp1` exactly. `map_localizer's` own relocalization initial pose defaults to (0,0,0) — i.e. it assumes the robot starts at the MAP

FRAME's own origin, NOT at wp1. If you actually place the robot at wp1's real-world spot before starting, pass that printed override to `localization.launch.py`, or VGLCP relocalization has to converge from however far (0,0) actually is from wp1 (potentially several metres) — risking a failed or wrong convergence instead of a clean one.

Run the picked waypoints

```
ros2 launch diy_waypoint_sequencer waypoint_sequencer.launch.py \
  waypoints_file:=maps/test_waypoints.yaml
```

6 · Still Open

- The custom controller stack is not yet wired into `challenge_master.launch.py` — currently launched standalone only. Master-launch integration is a separate, larger decision not made in this pass.
- Whether `diy_zone_nav` should eventually be deleted outright (vs. kept orphaned-but-present) was not decided — no urgency since it is already proven harmless to leave in place.
- `tf_transformations` is not installed in this sandbox (no sudo) — affects verifying this and the pre-existing `diy_motion_planner` package identically; not a new gap introduced by this work.

Full investigation and verification detail: [docs/reuse_plan_step1.md](#) Step 13.